

Implementing the Design

Objectives

After completing this lab, you will be able to:

- Implement the design
- Generate various reports and analyze the results
- Run static timing analysis
- Generate bitstream and verify the functionality in hardware

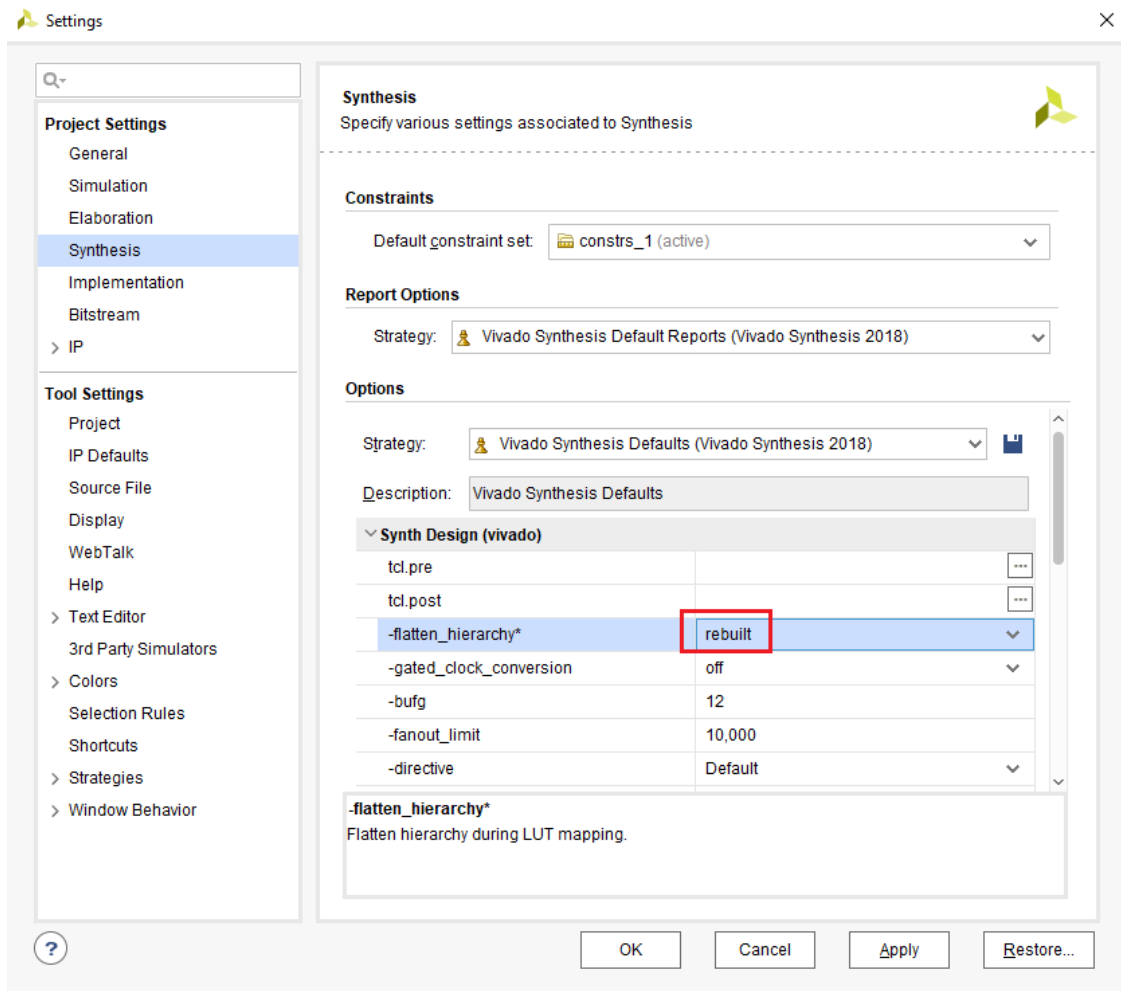
Steps

Open a Vivado Project using IDE

Launch Vivado and open the lab2 project. Save the project as lab3 in the <2018_2_zynq_labs> directory making sure that the create subdirectory option is selected. Set the flatten_hierarchy setting to rebuilt. Create new synthesis run naming it as synth_2.

<2018_2_zynq_labs> refers to the C:\xup\fpga_flow\2018_2_zynq_labs directory and <2018_2_zynq_sources> to the C:\xup\fpga_flow\2018_2_zynq_sources directory.

1. Start the Vivado if necessary and open the lab2 project (lab2.xpr) you created in the previous lab using the **Open Project** link in the Getting Started page.
2. Select **File > Project > Save As ...** to open the *Save Project As* dialog box. Enter **lab3** as the project name. Make sure that the *Create Project Subdirectory* option is checked, the project directory path is <2018_2_zynq_labs> and click **OK**.
3. Click on the **Settings** in the *Flow Navigator* pane.
4. Make sure that the *Synthesis > flatten_hierarchy* is set to **rebuilt**, which allows the design hierarchy to be preserved for synthesis, and then rebuilt which is more useful for design analysis because many logical references will be maintained.



Setting hierarchy to rebuilt

5. Click **OK**.

A Create New Run dialog box will appear asking you if a new run should be created. Click **Yes** and then **OK** to create the new run with **synth_2** name.

Synthesize the design. Generate the timing summary and analyze the design.

1. Click on **Run Synthesis** under the *Synthesis* tasks of the *Flow Navigator* pane.

The synthesis process will be run on the `uart_top.v` and all its hierarchical files. When the process is completed a *Synthesis Completed* dialog box with three options will be displayed.

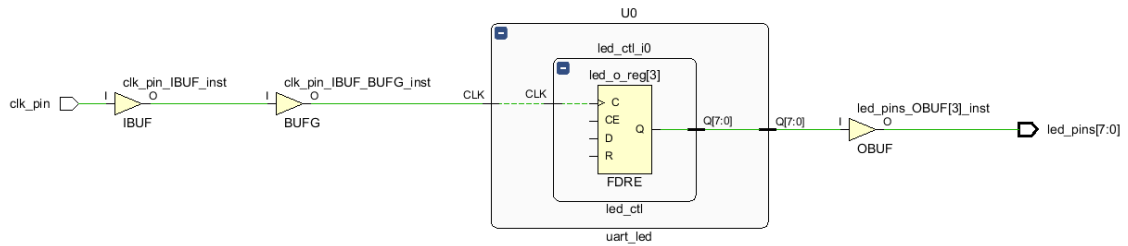
2. Select the *Open Synthesized Design* option and click **OK** as we want to look at the synthesis output.

3. Click on **Report Timing Summary** under the *Synthesized Design* tasks of the *Flow Navigator* pane.

4. Leave all the settings unchanged, and click **OK** to generate a default timing report, *timing_1*.

Source to clock port of the FDRE

- Finally, double-click on the input pin of IBUF to see the path between the clock input pin and the IBUF.



The schematic view of the source clock path

This corresponds to the Source Clock Path in the timing report.

Source Clock Path

Delay Type	Incr (ns)	Path (...)	Locati...	Netlist Resource(s)
(clock clk_pin rise edge)	(r) 8.000	8.000		
	(r) 0.000	8.000	Sit...16	clk_pin
net (fo=0)	0.000	8.000		clk_pin
			Sit...16	clk_pin_IBUF_inst/I
IBUF (Prop_ibuf I O)	(r) 1.451	9.451	Sit...16	clk_pin_IBUF_inst/O
net (fo=1, unplaced)	0.800	10.250		clk_pin_IBUF
				clk_pin_IBUF_BUFG_inst/I
BUFG (Prop_bufg I O)	(r) 0.101	10.351		clk_pin_IBUF_BUFG_inst/O
net (fo=59, unplaced)	0.584	10.935		U0/led_ctl_i0/CLK
FDRE				U0/led_ctl_i0/led_o_reg[3]/C

The source clock path for the PYNQ

Since the virtual clock is slower (12 ns) than the clk_pin period (8 ns), the data path delay includes the clock period of the clk_pin clock source.

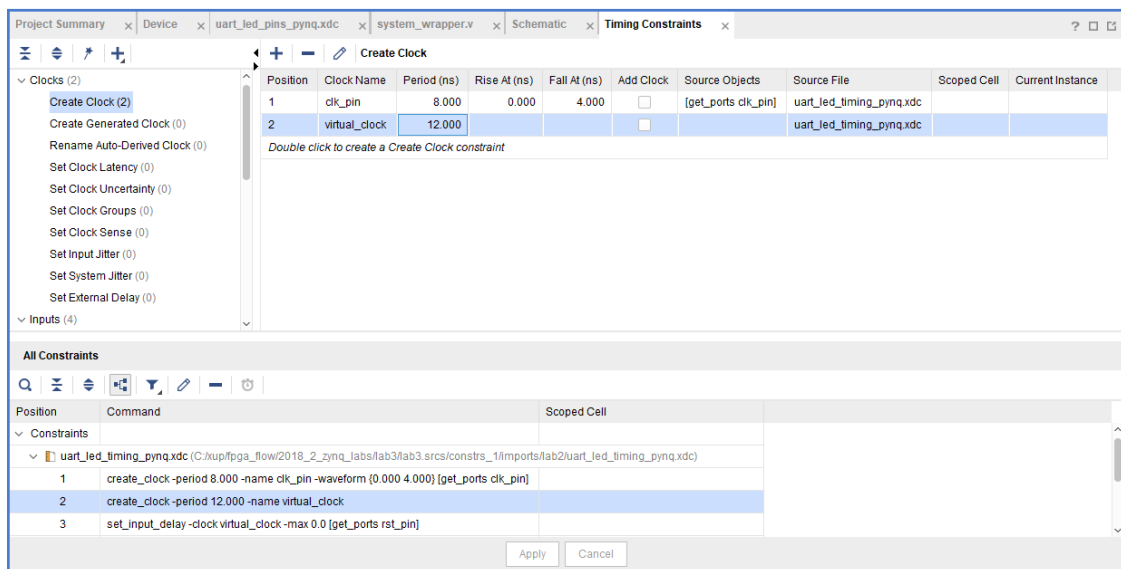
Source Clock Path					
Delay Type	Incr (ns)	Path (...)	Locati...	Netlist Resource(s)	
(clock clk_pin rise edge)	(r) 8.000	8.000			
	(r) 0.000	8.000	Sit...16	clk_pin	
net (fo=0)	0.000	8.000		clk_pin	
			Sit...16	clk_pin_IBUF_inst/I	
IBUF (Prop_ibuf I_O)	(r) 1.451	9.451	Sit...16	clk_pin_IBUF_inst/O	
net (fo=1, unplaced)	0.800	10.250		clk_pin_IBUF	
				clk_pin_IBUF_BUFG_inst/I	
BUFG (Prop_bufg I_O)	(r) 0.101	10.351		clk_pin_IBUF_BUFG_inst/O	
net (fo=59, unplaced)	0.584	10.935		U0/led_ctl_i0/CLK	
FDRE				U0/led_ctl_i0/led_o_reg[3]/C	
Data Path					
Delay Type	Incr (ns)	Path (...)	Locati...	Netlist Resource(s)	
FDRE (Prop_fdre C_Q)	(r) 0.478	11.413		U0/led_ctl_i0/led_o_reg[3]/Q	
net (fo=1, unplaced)	0.800	12.213		led_pins_OBUF[3]	
			Sit...14	led_pins_OBUF[3]_inst/I	
OBUF (Prop_obuf I_O)	(r) 3.756	15.969	Sit...14	led_pins_OBUF[3]_inst/O	
net (fo=0)	0.000	15.969		led_pins[3]	
			Sit...14	led_pins[3]	
Arrival Time		15.969			
Destination Clock Path					
Delay Type	Incr (ns)	Path (...)	Loca...	Netlist Resourc...	
(clock virtual...ck rise edge)	(r) 12.000	12.000			
ideal clock network latency	0.000	12.000			
clock pessimism	0.000	12.000			
clock uncertainty	-0.025	11.975			
output delay	-0.000	11.975			
Required Time		11.975			

Worst failing path for the PYNQ

Change the design constraint to constrain the virtual clock period to 8ns. Re-synthesize the design and analyze the results.

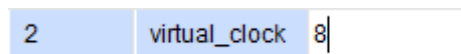
1. Click **Edit Timing Constraints** under the Synthesized Design.

The Timing Constraints GUI will appear, showing the design has two create clocks, four inputs, and one output constraints. It also shows the constraints in the text form in the All Constraints section.



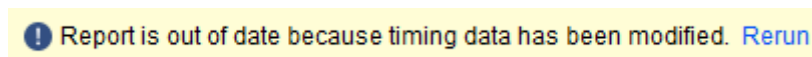
Timing Constraints showing 12 ns Virtual Clock period defined

- Click in the Period cell of the virtual_clock and change the period from 12 to 8



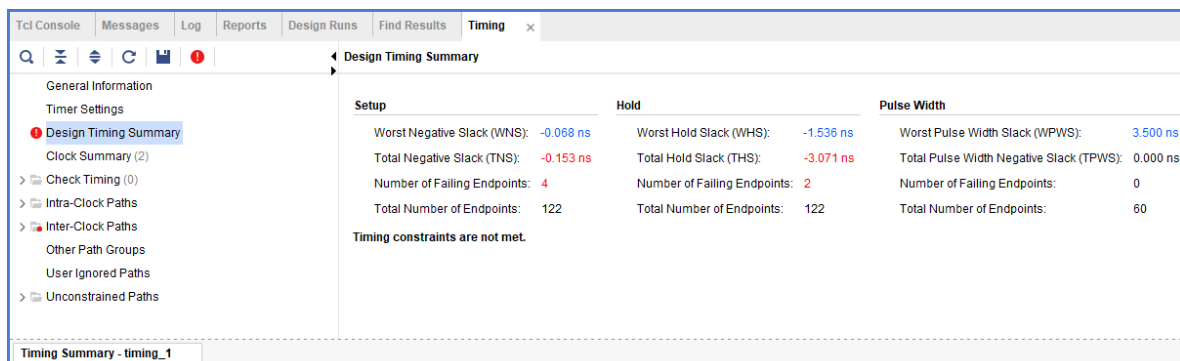
- Click **Apply**.

Note that since the timing constraint has changed, a warning message in the console pane is displayed to rerun the report.



- Click on **Rerun**.

Notice that setup timing violations are gone. However, there are still 2 failing paths for the Hold.



5. Click on the WHS link to see the paths.
6. Double-click on the first path to see the timing compositions. Notice that the clock path delay does not include the entire clock period.
7. Select **File > Constraints > Save...**
8. Click **Update**.
9. Click **OK** and then **Yes** to save the synthesized design.

Notice that the Synthesis Out-of-Date status is displayed on the top-right corner.

Implement the Design

Run the implementation after saving the synthesis run. Perform the timing analysis.

1. In the Design Runs tab, right-click on the synth_2 and select **Reset Runs**. Make sure the generated files are deleted. Click **Reset**.
2. Click the **Close Design** link in the status bar. If prompted, do not save anything.
3. Click on the **Run Implementation** in the *Flow Navigator* pane.
4. Click **OK** when prompted to run the synthesis first before running the implementation process.

When the implementation is completed, a dialog box will appear with three options.

5. Select the *Open Implemented Design* option and click **OK**.

View the amount of FPGA resources consumed by the design using Report Utilization.

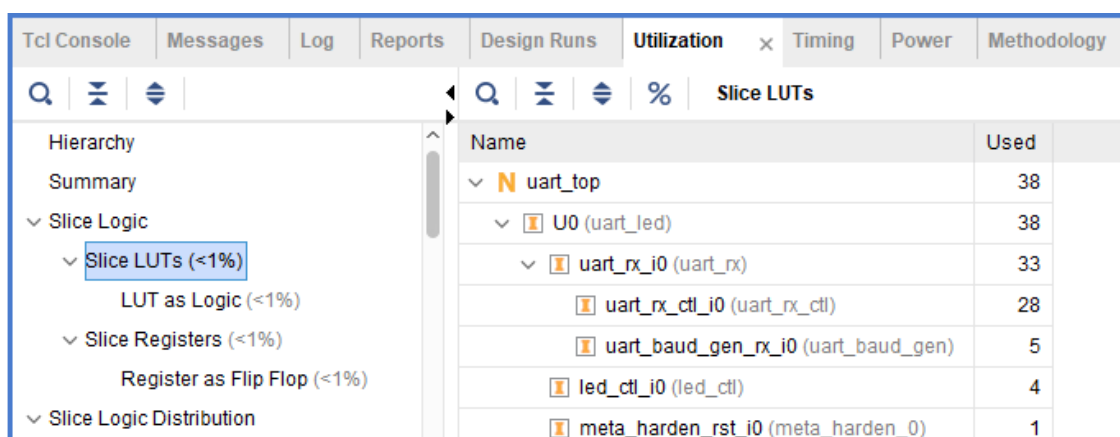
1. In the *Flow Navigator* pane, select IMPLEMENTATION > Open Implemented Design > Report Utilization.

The Report Utilization dialog box opens.

2. Click **OK**.

The utilization report is displayed at the bottom of the Vivado IDE. You can select any of the resources on the left to view its corresponding utilization.

3. Select Slice LUTs to view how much and which module consumes the resource.



The screenshot shows the Vivado IDE interface with the 'Utilization' tab selected. The left pane shows the 'Hierarchy' tree with 'Slice LUTs (<1%)' selected. The right pane displays a table of resource utilization for Slice LUTs.

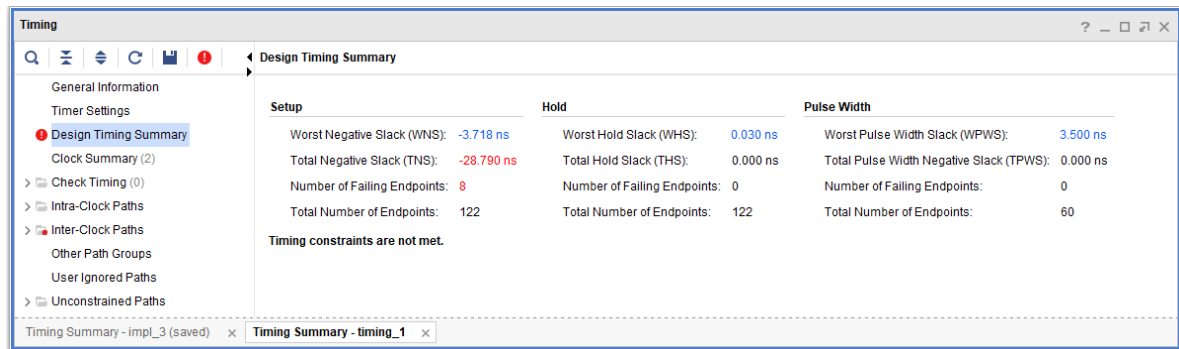
Name	Used
uart_top	38
U0 (uart_led)	38
uart_rx_i0 (uart_rx)	33
uart_rx_ctl_i0 (uart_rx_ctl)	28
uart_baud_gen_rx_i0 (uart_baud_gen)	5
led_ctl_i0 (led_ctl)	4
meta_harden_rst_i0 (meta_harden_0)	1

Generate a timing summary report

1. Click **Flow Navigator > IMPLEMENTATION > Open Implemented Design > Report Timing Summary**

The Report Timing Summary dialog box opens.

2. Leave all the settings unchanged and click **OK** to generate the report.



The timing summary report showing timing violations

3. Click on the WNS link to see a detailed report to determine the failing path entries.
4. Double-click on the first failing path to see why it is failing.

Source Clock Path

Delay Type	Incr (ns)	Path ...	Location	Netlist Resource(s)
(clock clk_pin rise edge)	(r) 0.000	0.000		
	(r) 0.000	0.000	Site: H16	clk_pin
net (fo=0)	0.000	0.000		clk_pin
			Site: H16	clk_pin_IBUF_inst/I
IBUF (Prop_ibuf I O)	(r) 1.451	1.451	Site: H16	clk_pin_IBUF_inst/O
net (fo=1, routed)	2.522	3.973		clk_pin_IBUF
			Site: BUF...TRL_X0Y16	clk_pin_IBUF_BUFG_inst/I
BUFG (Prop_bufg I O)	(r) 0.101	4.074	Site: BUF...TRL_X0Y16	clk_pin_IBUF_BUFG_inst/O
net (fo=59, routed)	2.057	6.131		U0/led_ctl_i0/CLK
FDRE			Site: SLICE_X113Y104	U0/led_ctl_i0/led_o_reg[3]/C

Data Path

Delay Type	Incr (ns)	Path (...)	Location	Netlist Resource(s)
FDRE (Prop_fdre C Q)	(r) 0.456	6.587	Site: SLICE_X113Y104	U0/led_ctl_i0/led_o_reg[3]/Q
net (fo=1, routed)	1.526	8.112		led_pins_OBUF[3]
			Site: M14	led_pins_OBUF[3]_inst/I
OBUF (Prop_obuf I O)	(r) 3.581	11.693	Site: M14	led_pins_OBUF[3]_inst/O
net (fo=0)	0.000	11.693		led_pins[3]
			Site: M14	led_pins[3]
Arrival Time		11.693		

Destination Clock Path

Delay Type	Incr (ns)	Path ...	Loca...	Netlist Resourc...
(clock virtual...ck rise edge)	(r) 8.000	8.000		
ideal clock network latency	0.000	8.000		
clock pessimism	0.000	8.000		
clock uncertainty	-0.025	7.975		
output delay	-0.000	7.975		
Required Time		7.975		

First failing path delays for the PYNQ

Compared to delays from the synthesis report, the net delays are actual delays (rather than an estimated figure). The data path delay is longer than the destination clock path delay giving a negative slack (violation). The data path delay is 11.693 ns for the PYNQ-Z2, the destination clock path is 7.975 ns and the negative slack is -3.718 ns.

At this point we can ignore this violation as the LED display change by a few nanoseconds won't be observable by human eyes. We can also change the output delay by -4 ns and make the timings meet.

5. Change the output delay by -4 ns and add the following min delay constraint in the uart_led_timing_pynq.xdc file.

```
set_output_delay -clock virtual_clock -4.000 [get_ports {led_pins[*]}]
set_output_delay -clock virtual_clock -min -1.000 [get_ports {led_pins[*]}]
```

6. Click on **Save File** button to save the modified constraint file and then click on the **Run Implementation** in the *Flow Navigator* pane to implement the project with new constraint file.

7. Click **OK** when prompted to run the synthesis first before running the implementation process.

When the implementation is completed, a dialog box will appear with three options.

8. Select the **Open Implemented Design** option and click **OK**.

9. Click **Flow Navigator > IMPLEMENTATION > Open Implemented Design > Report Timing Summary**

Observe that the timing violations of the Intra-clock paths are gone.

10. Expand the **Intra-Clock Paths** folder on the left, expand *clk_pin*, and select the Setup group to see the list of 10 worst case delays on the right side.

11. Double-click on the any path to see how that is made up of. Also right-click on it and select **Schematic**.

Click on the **Device** tab and see the highlighted path in the view.

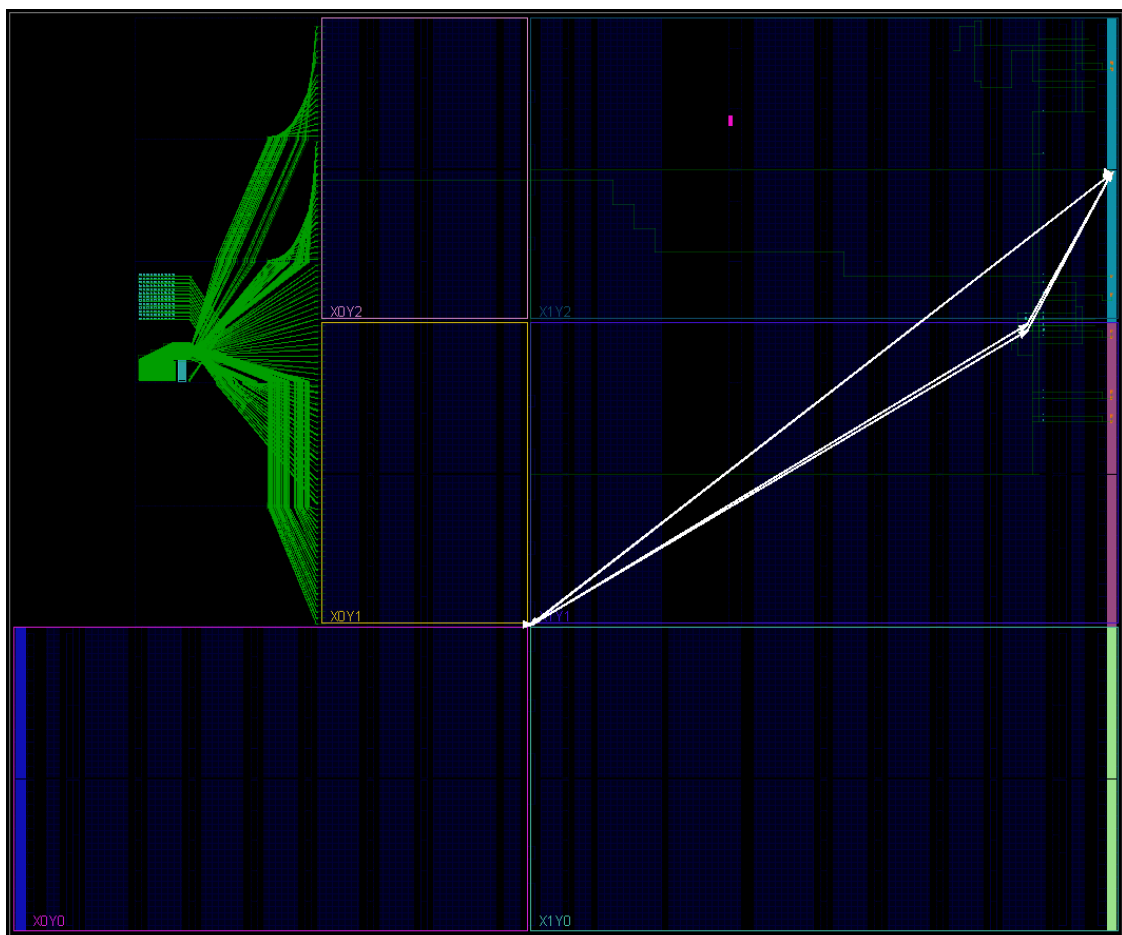
12. Select **Open Implemented Design > Report Clock Networks**.

13. Click **OK**.

The Clock Networks report will be displayed in the Console pane showing two clock net entries.

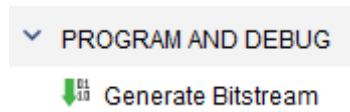
14. Select *clk_pin* entry and observe the selected nets in the Device view.

The clock nets are spread across multiple clock regions.



Generate the Bitstream

1. Click **Flow Navigator > PROGRAM AND DEBUG > Generate Bitstream**.



Generating the bitstream

2. The `write_bitstream` command will be executed (you can verify it by looking in the Tcl console).
3. Click **Cancel** when the bitstream generation is completed.

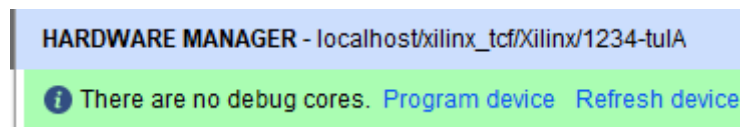
Verify the Functionality

Connect the board and power it ON. Open a hardware session, and program the FPGA.

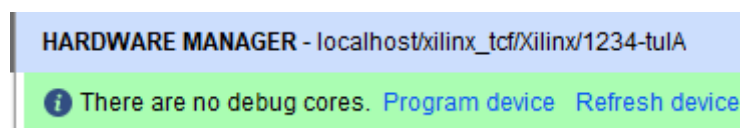
1. Make sure that the Micro-USB cable is connected to the JTAG PROG connector.
2. Select the *Open Hardware Manager* option.

The **HARDWARE MANAGER** window will open indicating “unconnected” status.

3. Click on the **Open target** link, then **Auto Connect** from the drop down menu.



4. The Hardware Session status changes from Unconnected to the server name and the device is highlighted. Also notice that the Status indicates that it is not programmed.
5. Select the device in the *Hardware Device Properties*, and verify that the **uart_top.bit** is selected as the programming file in the General tab.
6. Click on the FPGA device in the HARDWARE MANAGER window



7. Click on the **Program** button.

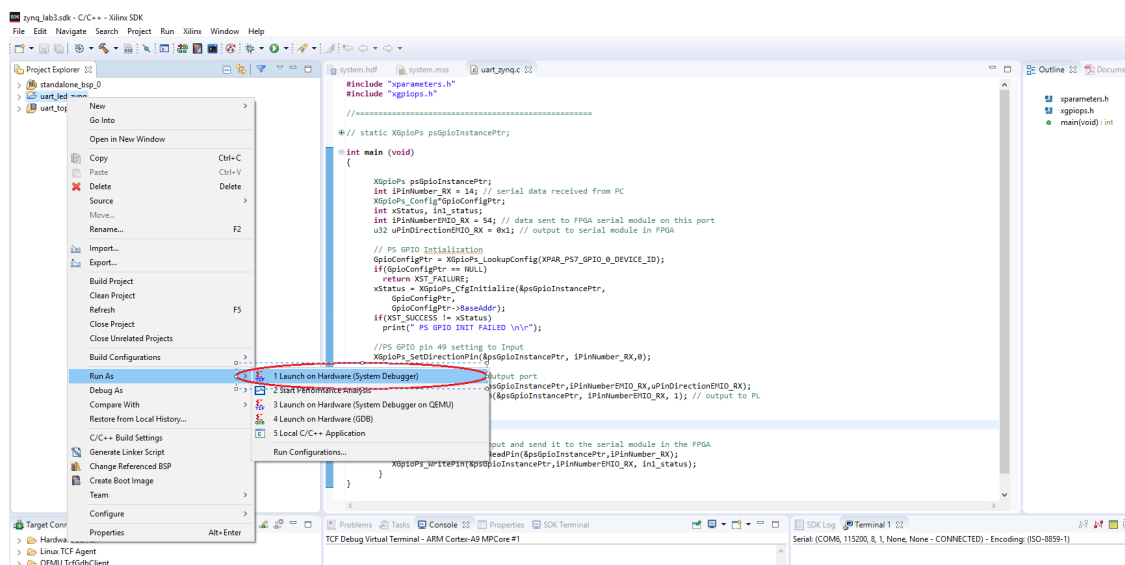
The programming bit file will be downloaded and the DONE light will be turned ON when the FPGA has been programmed.

Start a terminal emulator program such as TeraTerm or HyperTerminal. Select an appropriate COM port (you can find the correct COM number using the Control Panel). Set the COM port for 115200 baud rate communication.

1. Start a terminal emulator program such as TeraTerm or HyperTerminal.
2. Select an appropriate COM port (you can find the correct COM number using the Control Panel).
3. Set the COM port for 115200 baud rate communication.

Program FPGA, Start a SDK session, point it to the *C:/xup/fpga_flow/2018_2_zynq_sources/lab3/Pynq/lab3.sdk* workspace.

1. Open **SDK** by selecting **Start > Xilinx Design Tools > Xilinx SDK 2018.2**
2. In the **Select a directory as workspace** window, click on the browse button, browse to *C:/xup/fpga_flow/2018_2_zynq_sources/lab3/lab3.sdk* directory.
3. In the *Project Explorer*, right-click on the *uart_led_zynq*, select **Run As**, and then **Launch on Hardware (System Debugger)**.



Running the application

The program will be downloaded and the execution will begin.

4. In the serial terminal window, type in some characters in the terminal emulator window and see the corresponding ASCII equivalent bit pattern displayed on the LEDs.

Note: You will see the lower four bits on the LEDs

5. Press and hold BTN0 and see the the upper four bits are swapped with the lower four bits on the LEDs
6. When satisfied, close the terminal emulator program and power OFF the board.
7. Select **File > Close Hardware Manager**. Click **OK**.
8. Close the **Vivado** program by selecting **File > Exit** and click **OK**.

9. Close the **SDK** program by selecting **File > Exit** and click **OK**.

Conclusion

In this lab, you learned about many of the reports available to designers in the Vivado IDE. You had the opportunity to learn basic design analysis tools including the Schematic viewer, delay path properties and reports viewer, Device viewer, and selecting primitive parents. You also learned about the basic timing report options that are at your disposal. You verified the functionality in hardware by typing characters on the host machine and seeing the LED pattern changes.